DataBase-

Q1. What is indexing in databases?

Indexing in databases is a technique used to **speed up data retrieval** from a table.

Think of an index like the **index page of a book**:

- Without an index → you scan every page.
- With an index → you directly jump to the required page.

Similarly, a database index helps the DBMS quickly locate rows without scanning the entire table.

Example Without Index

Suppose you have a `users` table:

id	name	email
1	Amit	a@gmail.com
2	Rahul	r@gmail.com
3	Neha	n@gmail.com

Query:

```
SELECT * FROM users WHERE email = 'n@gmail.com';
```

Without indexing, the database performs a **full table scan**:

- Check row 1
- Check row 2
- Check row 3

This becomes slow for millions of rows.

Example With Index

Create index:

```
CREATE INDEX idx_email
ON users(email);
```

Now the database can directly find the required row much faster.

Q2. How Index Works Internally

Most databases use a **B-Tree** data structure for indexes.

It stores:

- Indexed column values
- Pointers to actual rows

This allows:

- Fast searching
- Fast sorting
- Efficient filtering

Types of Indexes

1. Primary Index

Automatically created on a primary key.

```
CREATE TABLE employee(
   id INT PRIMARY KEY,
   name VARCHAR(50)
);
```

Here, `id` gets indexed automatically.

---

2. Unique Index

Ensures values are unique.

```
CREATE UNIQUE INDEX idx_email
ON users(email);
```

No duplicate emails allowed.

---

3. Composite Index

Index on multiple columns.

```
CREATE INDEX idx_name_city
ON users(name, city);
```

Useful when queries use both columns together.

---

4. Clustered Index

Determines the physical order of data in the table.

- One clustered index per table
- Common in SQL Server

---

5. Non-Clustered Index

Separate structure pointing to actual data.

- Multiple allowed per table

Advantages of Indexing

✓ Faster `SELECT` queries
✓ Faster searching and sorting
✓ Improves JOIN performance
✓ Efficient filtering with `WHERE`

Disadvantages of Indexing

✗ Extra storage required
✗ Slower `INSERT`, `UPDATE`, `DELETE`
because indexes also need updating

Q3. When to Use Indexing

Use indexes on:

- Frequently searched columns
- Columns used in:
  - `WHERE`
  - `JOIN`
  - `ORDER BY`
  - `GROUP BY`

Example:

```
SELECT * FROM orders
WHERE customer_id = 10
ORDER BY order_date;
```

Indexes on `customer_id` and `order_date` help.

Q4. What is Foreign Key in DB?

A **Foreign Key** is a column (or group of columns) in one table that creates a relationship with the **Primary Key** of another table.

It is used to maintain **referential integrity**, meaning:

- You cannot insert invalid data
- Relationships between tables remain consistent

SQL Example-

```
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    name VARCHAR(50)
);

CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    amount DECIMAL(10,2),

    FOREIGN KEY (customer_id)
    REFERENCES customers(customer_id)
);
```

Q5. What Foreign Key Prevents

Suppose you try:

```
INSERT INTO orders VALUES (103, 10, 900);
```

If customer `10` does not exist in `customers`,
the database throws an error.

This prevents invalid relationships.

Referential Integrity-

Foreign keys ensure:

- Child table values must exist in parent table
- No orphan records

Example:

- An order cannot exist without a valid customer

Q6. What are Foreign Key Actions?

1. CASCADE

If parent row is deleted, child rows also delete automatically.

```
FOREIGN KEY (customer_id)
REFERENCES customers(customer_id)
ON DELETE CASCADE
```

---

2. SET NULL

Child foreign key becomes `NULL`.

```
ON DELETE SET NULL
```

---

3. RESTRICT / NO ACTION

Prevents deleting parent row if child rows exist.